



Assignment - 02 (DSA)  
Tutorial sheet - 3 (22BCON1068)

DATE: 27/9/23

PAGE NO.: 24

• Sec - A (5\*2=10)

Q1. Define stack. What are the applications of stack.

Ans. A stack is a widely used linear data structure in modern computers in which insertion and deletions of an element can occur only at one end, i.e., top of the stack. It follows the last-in-first-out (LIFO) principle. This means that the last element added to the stack is the first element to be removed. Stacks can be implemented using arrays or linked lists. The applications of stack are -

- (i) Function call management.
- (ii) Backtracking
- (iii) Memory Management
- (iv) Expression Evaluation
- (v) Browser History.

Q2. List all the basic operations that can be performed on a stack.

Ans. The basic operation that can be performed on a stack are-

- (i) push(): Adds one element to the top of the stack.
- (ii) pop(): Removes the top element from the stack and returns it.
- (iii) top(): Returns the top element of the stack without removing it.
- (iv) isEmpty(): Checks if the stack is empty.
- (v) isFull(): Checks if the stack is full.
- (vi) size(): Return the size of stack.

Q3. What is Overflow and underflow in stack?

Ans. Stack overflow and underflow are two errors that can occur when using a stack data structure.

- Overflow: Stack overflow occurs when a program tries to push an element onto a stack that is already full. This can happen if the stack has a limited size or if the program is using recursion to call



itself to many times.

- Underflow: Stack underflow occurs when a program tries to POP an element from an empty stack. This can happen if the program has already popped all of the element from the stack or if the program is using recursion and has not yet pushed any element onto the stack.

Q4. What is the postfix expression for the corresponding infix expression?

$$a + b * c + (d * e).$$

Ans

$$abc * + de * +$$

Q5. The equivalent infix expression for given postfix expression  $abc + de / * -$  is?

Ans

$$\text{Infix} \rightarrow a(b+c)(d/e) * -$$

$$\Rightarrow a(b+c) * (d/e) -$$

$$\Rightarrow a - (b+c) * (d/e) \quad \text{Ans.}$$

- Section-B ( $3 * 7 = 21$ )

Q1. Differentiate iteration and recursion with example.

Ans

Iteration is defined as the repetition of computational or mathematical procedure that continues until the controlling condition becomes false. It uses repetition structure. If the loop condition test never becomes false, then an infinite loop occurs with iteration.

Recursion is defined as a process in which a function calls itself repeatedly. Recursion uses selection structure. If the recursion step does not reduce the problem in a manner that converges on some condition, called base condition then an infinite recursion occurs. An infinite recursion can crash the system.

→ Diff? between Iteration & Recursion.

| Iteration                                                                          | Recursion                                               |
|------------------------------------------------------------------------------------|---------------------------------------------------------|
| 1. There is a repeated implementation of the bunch of instruction.                 | Recursion is the method of calling a function itself.   |
| 2. The code size is large as compare to recursion.                                 | The code or size is smaller than iteration's code size. |
| 3. it is fast as compared to recursion.                                            | The speed of recursion is slow.                         |
| 4. it has low time complexity.                                                     | The time complexity is higher as compared to iteration. |
| 5. it requires less memory.                                                        | it requires more memory.                                |
| 6. iteration uses the repetition structure.                                        | Recursion uses the selection structure.                 |
| 7. <del>iteration</del> iteration terminates when the condition in the loop fails. | Recursion terminates when the base case is met.         |
| 8. Example of iterative function.                                                  | Example of recursive function.                          |

```

public class IterationEx {
    public static void main (
        String[] args) {
        for (int i = 1; i <= 10; i++)
        {
            System.out.println(i);
        }
    }
}

```

```

public class RecursionEx {
    public static void PrintNumber(int n)
    {
        if (n == 0) {
            return;
        } else {
            System.out.println(n);
            PrintNumber(n-1);
        }
    }

    public static void main (String[] args) {
        PrintNumber(10);
    }
}

```



Q2. Convert following infix expressions into prefix expressions:

(1)  $(A * B) + (C / D) - (D + E)$

Ans. Prefix: Reverse the expression i.e.,  $(E + D) - (D / C) + (B * A)$

| Expression | Stack   | Prefix           |
|------------|---------|------------------|
| (          | (       |                  |
| E          | (       | E                |
| +          | ( +     | E                |
| D          | ( +     | ED               |
| )          |         | ED +             |
| -          | -       | ED +             |
| (          | -(      | ED +             |
| D          | -(      | ED + D           |
| /          | -( /    | ED + D           |
| C          | -( /    | ED + DC          |
| )          | -       | ED + DC /        |
| +          | - +     | ED + DC /        |
| (          | - + (   | ED + DC /        |
| B          | - + (   | ED + DC / B      |
| *          | - + ( * | ED + DC / B      |
| A          | - + ( * | ED + DC / BA     |
| )          | - +     | ED + DC / BA *   |
|            |         | ED + DC / BA * + |

Reverse the expression  $\Rightarrow - + * AB / CD + DE$  Ans

(b)  $((A - B) + D) / ((E + F) * G)$

Reverse the expression:  $((G * (F + E)) / D + (B - A))$

| Expression | Stack  | Prefix |
|------------|--------|--------|
| (          | (      |        |
| (          | ((     |        |
| G          | ((     | G      |
| *          | (( *   | G      |
| (          | (( * ( | G      |

|   |          |                       |
|---|----------|-----------------------|
| F | (( * (   | G F                   |
| + | (( * ( + | G F                   |
| E | (( * ( + | G F E                 |
| ) | (( *     | G F E +               |
| ) | (        | G F E + *             |
| / | ( /      | G F E + *             |
| D | ( /      | G F E + * D           |
| + | ( +      | G F E + * D /         |
| ( | ( + (    | G F E + * D /         |
| B | ( + (    | G F E + * D / B       |
| - | ( + (-   | G F E + * D / B       |
| A | ( + (-   | G F E + * D / B A     |
| ) | ( +      | G F E + * D / B A -   |
| ) |          | G F E + * D / B A - + |

Reverse of expression:  $+ - A B / D * + E F G$  Ans.

Q3. Differentiate bet<sup>n</sup> Infix, Prefix, and Postfix Notations.

Ans. Infix Notation: it is the most common notation used in mathematics and is familiar to most people. It is of the form  $\langle \text{operand} \rangle \langle \text{operator} \rangle \langle \text{operand} \rangle$ . As the name suggests, the operator is fixed inside bet<sup>n</sup> the operands. Such expressions are easy to understand and evaluate for human beings. However, computer finds it difficult. To pass information is needed about operator precedence and associativity rules and brackets which override these rules.

Eg:- ~~A B~~  $(A * B) / C$

Infix notation is the most common way to write mathematical expression. It uses parentheses to group operand and operators.



- Prefix Notation: The Prefix notation as the name suggests has the operator placed before the operand is specified. it is of the form  $\langle \text{operator} \rangle \langle \text{operand} \rangle \langle \text{operand} \rangle$ . It works entirely in same manner as Prefix expression. For evaluation, we evaluate it from left to right.  
Eg:  $* + AB - CD$

- Postfix Notation: In this notation, the operator placed right after the two operands. it is of the form  $\langle \text{operand} \rangle \langle \text{operand} \rangle \langle \text{operator} \rangle$ . It itself determines the precedence of operators i.e; the operator which occurs first operates on the operands.  
Eg:  $AB + CD - *$

- Key differences:-

| Notation | Operators        | Operands      |
|----------|------------------|---------------|
| Infix    | Between operands | Left to Right |
| Prefix   | Before operands  | Right to Left |
| Postfix  | After operands   | Left to Right |

- RE Section - C (3x11=33)

Q1 - Explain tower of Hanoi with stack ADT.

Ans - The tower of Hanoi is a mathematical puzzle. It consists of three ~~tower~~ towers (pegs) and a set of disks of different sizes. The disks can be stacked onto the pegs, with the largest disk at the bottom and smallest disk at the top. The objective is the, to move the entire stack from one tower known as source tower to another tower known as target tower. Using the third tower called the auxiliary tower as temporary storage. The puzzle follows three simple rules.

- (i) Only one disk can be moved among the towers at any given time.
- (ii) Only the "top" disk can be removed.
- (iii) No larger disk can be placed over a smaller disk.

# Implementation of Tower of Hanoi Problem in Java using Stack.

```

→ import java.io.*;
import java.math.*;
import java.util.*;

public class GFG {
    static void towerOfHanoi (int n, char from-rod,
                             char to-rod, char aux-rod)
    {
        if (n == 0) {
            return;
        }
        towerOfHanoi (n-1, from-rod, aux-rod, to-rod);
        SOP("Move disk " + n + " from rod " + from-rod + " to rod "
            + to-rod);
        towerOfHanoi (n-1, aux-rod, to-rod, from-rod);
    }

    public static void main (String args[]) {
        int N = 3;
        towerOfHanoi (N, 'A', 'C', 'B');
    }
}

```

OUTPUT: Move disk 1 from rod A to rod C

Move disk 2 from rod A to rod B

1 1 1 1 1

1 1 1 1 1

move disk 1 from rod A to rod C



Q2. Write an algorithm to convert an infix expression to its corresponding postfix expression, using a stack.

Ans. Here an algorithm to convert an infix expression to its corresponding postfix expression, using stack:

- (i) initialize a stack to store operators.
- (ii) Scan the infix expression from left to right.
- (iii) If the scanned character is an operand, append it to the postfix expression.
- (iv) If the scanned character is an operator:
  - If the stack is empty or the top element of the stack is a left parenthesis, push the operator onto the stack.
  - Otherwise, if the precedence of the scanned operator is less than or equal to the precedence of the operator at the top of the stack, pop the operator from the stack and append it to the postfix expression. Then, push the scanned operator onto the stack.
- (v) If the scanned character is a right parenthesis:
  - Pop the operators from the stack and append them to the postfix expression until a left parenthesis is encountered.
- (vi) After scanning the entire expression, pop all the operators from the stack and append them to the postfix expression.

# Code:

```
import java.util.Stack;

public class InfixToPostfixConverter {
    private static final int PREC_ADD = 1;
    private static final int PREC_SUB = 1;
    private static final int PREC_MUL = 2;
```

```
private static final int PREC_DIV = 2;

public static String convert(String infixExpression) {
    Stack<Character> stack = new Stack<>();
    StringBuilder postfixExpression = new StringBuilder();

    for (char character : infixExpression.toCharArray()) {
        if (Character.isAlphabetic(character)) {
            postfixExpression.append(character);
        } else if (character == '(') {
            stack.push(character);
        } else if (character == ')') {
            while (!stack.isEmpty() && stack.peek() != '(') {
                postfixExpression.append(stack.pop());
            }
            stack.pop();
        } else {
            while (!stack.isEmpty() && Precedence(stack.peek()) >=
                Precedence(character)) {
                postfixExpression.append(stack.pop());
            }
            stack.push(character);
        }
    }

    while (!stack.isEmpty()) {
        postfixExpression.append(stack.pop());
    }

    return postfixExpression.toString();
}

private static int Precedence(char operator) {
    switch (operator) {
        case '+':
```



```

case '-' :
    return PREC - ADD;
case '*' :
case '/' :
    return PREC - MUL;
default :
    return 0;
}

```

P.S.V.M (String C7 args)E

String InfixExpression = "A+B\*C-D/E";

String postfixExpression = convert(InfixExpression);

SOP("Infix expression:" + InfixExpression);

SOP("Postfix expression:" + postfixExpression);

}

OUTPUT: Infix expression: A+B\*C-D/E

Postfix expression: AB\*C-DE/

Q3. Convert following infix expression into postfix expression using a stack:

$((a+b)/d) - ((e-f)+g)$ .

Ans.

Expression

Stack

Postfix

(

(

(

((

a

((

a

+

((+

a

b

((+

ab

)

(

ab+

/

(/

ab+

d

(/

ab+d/

)

ab+d/

-

-

ab+d/

|   |    |             |
|---|----|-------------|
|   |    | ab+d/       |
| ( | -( | ab+d/       |
| ( | -( | ab+d/e      |
| e | -( | ab+d/e      |
| - | -( | ab+d/ef     |
| f | -( | ab+d/ef-    |
| ) | -( | ab+d/ef-    |
| + | -( | ab+d/ef-g   |
| g | -( | ab+d/ef-g+  |
| ) | -  | ab+d/ef-g+- |

Postfix expression:  $ab+d/ef-g+-$

OR

# Code:

```

import java.util.Stack;

public class InfixToPostfixConverter {
    private static final int PREC_ADD = 1;
    private static final int PREC_SUB = 1;
    private static final int PREC_MUL = 2;
    private static final int PREC_DIV = 2;

    public static String convert(String infixExpression) {
        Stack<Character> stack = new Stack<>();
        StringBuilder postfixExpression = new StringBuilder();

        for (char character : infixExpression.toCharArray()) {
            if (Character.isAlphabetic(character)) {
                postfixExpression.append(character);
            } else if (character == '(') {
                stack.push(character);
            } else if (character == ')') {
                postfixExpression.append(stack.pop());
            }
        }
    }
}

```



```

    stack.pop();
} else {
    while (!stack.isEmpty() && precedence(stack.peek()) >=
        precedence(character)) {
        postfixExpression.append(stack.pop());
    }
    stack.push(character);
}
}

```

```

private static int precedence(char operator) {
    switch (operator) {
        case '+':
        case '-':
            return PREC_ADD;
        case '*':
        case '/':
            return PREC_MUL;
        default:
            return 0;
    }
}

public void (String args[]) {

```

```

    String infixExpression = "((a+b)/d)-((e-f)+g)";
    String postfixExpression = convert(infixExpression);
    SOP("Infix expression: " + infixExpression);
    SOP("Postfix expression: " + postfixExpression);
}
}

```

Output: Infix expression: ((a+b)/d)-((e-f)+g)  
 Postfix expression: ab+d/ef-g+-

The end.

• Section-A ( $5 \times 2 = 10$ )

Q1. Why stack is called a LIFO data structure?

Ans. A stack is called a LIFO (Last In, First Out) data structure because elements are added and removed from the top of the stack. This means that the last element added to the stack is the first element to be removed.

Q2. The time complexity of converting a Prefix notation to infix notation is ---.

Ans. The time complexity of converting a Prefix notation to infix notation is  $O(n)$ , where  $n$  is the length of the Prefix expression.

Q3. What would be the solution of the given Prefix notation?

~~Ans.~~  $- + 5 / 10 5 5$

Ans.

$- + 5 / 10 5 5$

$\Rightarrow + 5 2 5$

$\Rightarrow - 7 5$

$\Rightarrow 2$

Ans.

Q4. State the different ways of representing expressions.

Ans. The three main ways of representing expressions are:

- Infix: Operators between Operands.

ex:  $(1+2)$

- Postfix: Operators after Operands.

ex:  $1 2 +$

- Prefix: Operators before Operands.

ex:  $+ 1 2$



Q5. What is the number of moves required in the Tower of Hanoi Problem for  $k$  disks?

Ans. The number of moves required in the Tower of Hanoi Problem =  $2^N - 1$

Here  $N = k$

So for  $k$  disks is  $2^k - 1$ .

This can be proven by using a recursive algorithm. The algo. works by moving the smallest disk to the destination peg.

• Section- B ( $3 \times 7 = 21$ )

Q1. Derive the equivalent infix expression and value for the Postfix form  $12 + 3 * 4 5 * -$ .

Ans.

$12 + 3 * 4 5 * -$

1.  $[1]$

2.  $[1, 2]$

3.  $+ \Rightarrow [3]$

4.  $[3, 3]$

5.  $* \Rightarrow [9]$

6.  $[9, 4]$

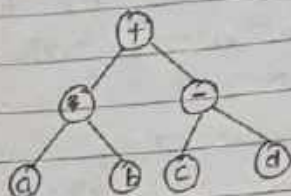
7.  $[9, 4, 5]$

8.  $* \Rightarrow [9, 20]$

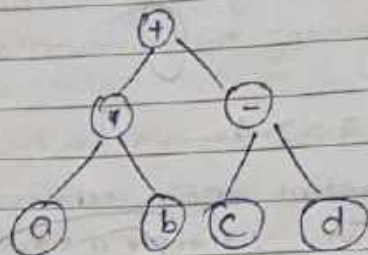
9.  $- \Rightarrow [11]$

Ans = 11

92. from the given Expression tree, find out the Postfix expression.



Ans.



=>

a b \* c d - + => Postfix expression

Ans.

93. What is enqueue dequeue in stack? Explain with example.

Ans. Enqueue and dequeue are operation that are not supported by stack. Stacks are LIFO (last-in-first-out) data structures, which means that element can only be added and removed from the top of the stack. Enqueue and dequeue operations are associated with queues.



which are FIFO (First-in-First-Out) data structures.

A queue is a linear data structure in which elements are added to the back of the queue and removed from the front of the queue.

Enqueue is the operation of adding an element to the back of a queue.

Dequeue is the operation of removing an element from the front of a queue.

Enqueue and dequeue operation are useful for implementing a variety of algorithms, such as the following:

- Breadth-First search: A breadth-first search is an algorithm for travelling a graph. It works by starting at a root node and exploring all of the nodes that are adjacent to the root node. Then, it explores all of the nodes that are adjacent to the nodes that were explored in the previous step, and so on.
- Job scheduling: A job scheduling algorithm is used to determine the order in which jobs should be executed on a computer system. Job scheduling algorithms often use queues to keep track of the jobs that need to be executed.
- Operating system memory management: OSMM algorithm uses queues to keep track of the memory that is available to be allocated to processes.

eg: Repeat all the steps till  $TDS = \text{max} - 1$

step1: check to see if the stack is full

$$TDS = \text{MAX} - 1$$

write "overflow" or "exit"

step2: •  $TDS = TDS + 1$

•  $S[TDS] = \text{val}$

• Section - C (2x11=22)

- (1) What do you understand by stack overflow and underflow? Describe the evaluation of postfix expressions with an example. What is the time complexity of enqueue and dequeue in stack.

Ans.

- Stack overflow occurs when a stack is filled to its capacity and no more element can be added.
- Stack underflow occurs when a stack is empty and no more elements can be removed.

- Evaluation of postfix expressions: Postfix expressions are evaluated using a stack. The stack is used to store the operands of the expression as they are encountered. When an operator is encountered, the top operands of the top of the stack are popped and the operator is applied to them. The result is then pushed onto the stack. The process is repeated until the end of the expression is reached.

ex: (3 4 \* 5 2 + -) :

1. [3]
2. [3, 4]
3. \*  $\Rightarrow$  [1, 2]
4. [12, 5]
5. [12, 5, 2]
6. +  $\Rightarrow$  [12, 7]
7. -  $\Rightarrow$  [-5]

Solution = -5 Ans



- Time Complexity of enqueue and dequeue in stack:

The time complexity of enqueue and dequeue operation in a stack is  $O(1)$ . This is because both operations can be implemented in constant time using a single pointer.

To enqueue an element, we simply push the element onto the top of the stack. To dequeue an element, we simply pop the element from the top of the stack.

Q2. How Insertion and deletion works in stack? If the sequence of operations - Push(1), Push(2), Pop, Push(1), Push(2), Pop, Pop, Pop, Push(2), Pop are performed on a stack. What will be the sequence of popped out values.

Ans. Insertion in a stack is performed by pushing an element onto the top of the stack. This can be done using a pointer to the top of the stack. When an element is pushed onto the stack, the pointer is moved up to point to the new element.

```
ex: public object POP(Stack stack)
    {
        return stack.POP();
    }
```

- Deletion: Deletion in a stack is performed by popping an element from the top of the stack. This can be done by using the pointer to the top of the stack. When an element is popped from the stack, the pointer is moved down to point to the next element in the stack.

- (1) Push(1)  $\Rightarrow$  [1]
- (2) Push(2)  $\Rightarrow$  [1, 2]
- (3) POP  $\Rightarrow$  [1]
- (4) Push(1)  $\Rightarrow$  [1, 1]
- (5) POP Push(1)  $\Rightarrow$  [1, 1, 2]
- (6) POP  $\Rightarrow$  [1, 1]
- (7) POP  $\Rightarrow$  [1]
- (8) POP  $\Rightarrow$  [ ]
- (9) Push(2)  $\Rightarrow$  [2]
- (10) POP  $\Rightarrow$  [ ]

Sequence of the popped out values : 2, 2, 1, 1, 2

Q3. (i) The following postfix expression with single digit operand is evaluated using a stack. Find output  
 $7\ 3\ 4\ ^\wedge\ /\ 3\ 4\ * + 6\ 2\ ^\wedge -$

Ans

- (i) [7]
- (ii) [7, 3]
- (iii) [7, 3, 4]
- (iv)  $\wedge \Rightarrow 3^4 \Rightarrow 81 \Rightarrow$  [7, 81]
- (v)  $/ \Rightarrow 7/81 \Rightarrow$  [7/81]
- (vi)  $3 \Rightarrow$  [7/81, 3]
- (vii)  $4 \Rightarrow$  [7/81, 3, 4]
- (viii)  $* \Rightarrow$  [7/81, 12]
- (ix)  $+$   $\Rightarrow$  [7/81 + 12]
- (x)  $6 \Rightarrow$  [7/81 + 12, 6]
- (xi)  $2 \Rightarrow$  [7/81 + 12, 6, 2]
- (xii)  $* \Rightarrow$  [7/81 + 12, 12]
- (xiii)  $- \Rightarrow$  [7/81]

A



(ii) Convert Infix expression into Postfix expression using stack  $(P+Q) * (R * S - T) * U / V$  is ?

Ans:

| Symbol | Operand | Stack | Postfix        |
|--------|---------|-------|----------------|
| (      |         | (     |                |
| P      |         | (     | P              |
| +      |         | ( +   | P              |
| Q      |         | ( +   | PQ             |
| )      |         |       | PQ+            |
| *      |         | *     | PQ+            |
| (      |         | *(    | PQ+            |
| R      |         | *(    | PQ+R           |
| *      |         | *(    | PQ+R           |
| S      |         | *(    | PQ+RS          |
| -      |         | *(-   | PQ+RS          |
| T      |         | *(-   | PQ+RS T        |
| )      |         |       | PQ+RS T-       |
| *      |         | *     | PQ+RS T-*      |
| U      |         | *     | PQ+RS T-* U    |
| /      |         | /     | PQ+RS T-* U/   |
| V      |         | /     | PQ+RS T-* U*V/ |

Postfix Expression :  $PQ+RS T-* U*V/$

Ans

*Handwritten signature*  
16/10/2023

(22BCON11068)